21CSE451T

PATTERN RECOGNITION TECHNIQUES

UNIT-3

NON-PARAMETRIC TECHNIQUES

- **Density Estimation**
- **Parzen Windows**
- **K-Nearest Neighbor Estimation**
- **The Nearest Neighbor Rule**
- **Metrics and Nearest Neighbor Classification**
- **Fuzzy Classification**
- **Reduced Column Energy Networks**
- **Approximations by Series Expansions**

NON-PARAMETRIC TECHNIQUES

- Treat supervised learning under the assumption that the forms of the underlying density functions were known. In most pattern recognition applications this assumption is suspect; the common parametric forms rarely fit the densities actually encountered in practice.
- In particular, all of the classical parametric densities are unimodal (have a single local maximum), whereas many practical problems involve multimodal densities.
- Furthermore, our hopes are rarely fulfilled that a high-dimensional density might be accurately represented as the product of one-dimensional functions.
- So, *nonparametric* procedures that can be used with arbitrary distributions and without the assumption that the forms of the underlying densities are known.
- There are several types of nonparametric methods of interest in pattern recognition.
    - One consists of procedures for estimating the density functions $p(x|w_j)$ from sample patterns. If these estimates are satisfactory, they can be substituted for the true densities when designing the classifier.
    - Anotherr consists of procedures for directly estimating the *a posteriori* probabilities $P(w_j|x)$. This is closely related to nonparametric design procedures such as the nearest-neighbor rule, which bypass probability estimation and go directly to decision functions.

DENSITY ESTIMATION

The basic ideas behind many of the methods of estimating an unknown probability density function are very simple, although rigorous demonstrations that the estimates converge require

considerable care. The most fundamental techniques rely on the fact that the probability P that a vector x will fall in a region R is given by:

$$P = \int_{\mathcal{R}} p(\mathbf{x}')\, d\mathbf{x}'$$

Thus P is a smoothed or averaged version of the density function $p(x)$, and we can estimate this smoothed value of p by estimating the probability P. Suppose that n samples X i , . . . , x,, are drawn independently and identically distributed (i.i.d.)according to the probability law p(x). Clearly, the probability that k of these n fall
in R is given by the binomial law:

$$P_k = \binom{n}{k} P^k (1 - P)^{n-k}$$

and the expected value for k is:

$$\mathcal{E}[k] = nP$$

Moreover, this binomial distribution for k peaks very sharply about the mean, so that we expect that the ratio k/n will be a very good estimate for the probability P, and hence for the smoothed density function. This estimate is especially accurate when n is very large. If we now assume that $p(x)$ is continuous and that the region R is so
small that p does not vary appreciably within it, we can write:

$$\int_{\mathcal{R}} p(\mathbf{x}')\, d\mathbf{x}' \simeq p(\mathbf{x})V,$$

where x is a point within R and V is the volume enclosed by R. Estimate for $p(x)$:

$$p(\mathbf{x}) \simeq \frac{k/n}{V}$$

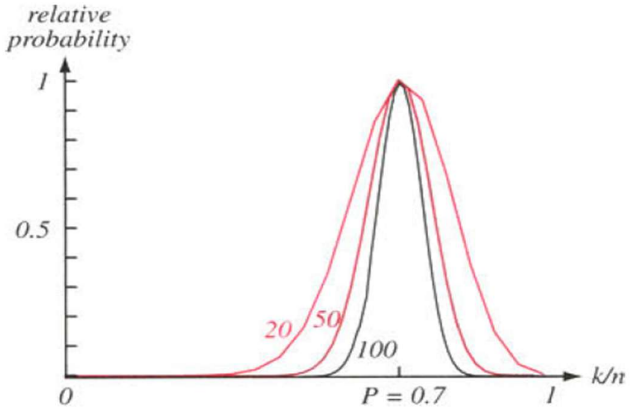

FIGURE 4.1. The relative probability an estimate will yield a particular value for the probability density, here where the true probability was chosen to be 0.7. Each curve is labeled by the total number of patterns n sampled, and is scaled to give the same maximum (at the true probability). The form of each curve is binomial. For large n, such binomials peak strongly at the true probability. In the limit n -> (∞), the curve approaches a delta function, and we are guaranteed that our estimate will give the true probability.

Problem:

- If we fix the volume V and take more and more training samples, the ratio k/n will converge (in probability) as desired, but then we have only obtained an estimate of the space-averaged value of $p(x)$:

$$\frac{P}{V} = \frac{\int_{\mathcal{R}} p(\mathbf{x}')\, d\mathbf{x}'}{\int_{\mathcal{R}} d\mathbf{x}'}$$

- If we want to obtain $p(x)$ rather than just an averaged version of it, we must be prepared to let V approach zero.
- If we fix the number n of samples and let V approach zero, the region will eventually become so small that it will enclose no samples, and our estimate $p(x) \sim 0$ will be useless.
- If by chance one or more of the training samples coincide at x, the estimate diverges to infinity, which is equally useless.
- From a practical standpoint, we note that the number of samples is always limited. Thus, the volume V cannot be allowed to become arbitrarily small. If this kind of estimate is to be used, one will have to accept a certain amount of variance in the ratio k/n and a certain amount of averaging of the density $p(x)$.

How these limitations can be circumvented if an unlimited number of samples is available?

Suppose we use the following procedure. To estimate the density at x, we form a sequence of regions $R_1, R_2, \ldots$, containing x—the first region to be used with one sample, the second with two, and so on. Let V_n be the volume of R_n, k_n be the number of samples falling in R_n and $p_n(x)$ be the nth estimate for $p(x)$:

$$p_n(\mathbf{x}) = \frac{k_n/n}{V_n}$$

If $p_n(x)$ is to converge to $p(x)$, three conditions appear to be required:

- $\lim_{n \to \infty} V_n = 0$
- $\lim_{n \to \infty} k_n = \infty$
- $\lim_{n \to \infty} k_n/n = 0.$

- The first condition assures us that the space averaged P/V will converge to $p(x)$, provided that the regions shrink uniformly and that $p(.)$ is continuous at x.
- The second condition, which only makes sense if $p(x) \neq 0$, assures us that the frequency ratio will converge (in probability) to the probability P.
- The third condition is clearly necessary if $p_n(x)$, is to converge at all.

There are two common ways of obtaining sequences of regions that satisfy these conditions:

- One is to shrink an initial region by specifying the volume V_n as some function of n, such as $V_n = 1/\sqrt{n}$. It then must be shown that the random variables k_n and kn/n behave properly or, more to the point, that $p_n(x)$ converges to $p(x)$. This is basically the Parzen-window method.

- The second method is to specify k_n as some function of n, such as $k_n = \sqrt{n}$. Here the volume V_n is grown until it encloses k_n neighbors of x. This is the k_n -nearest-neighbor estimation method.

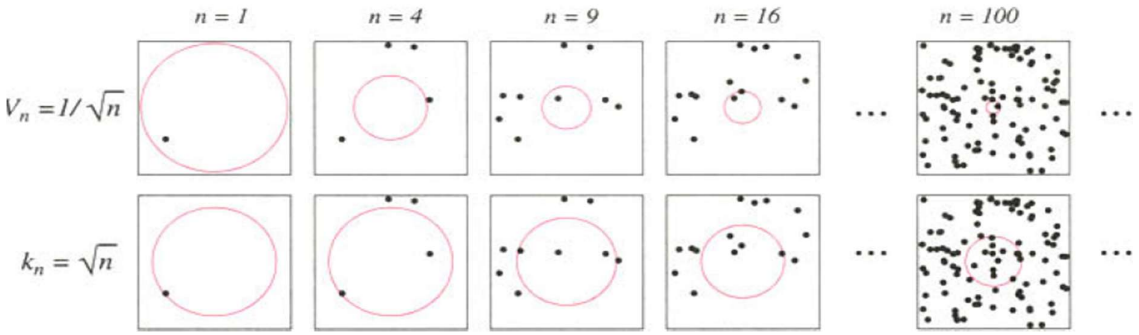

FIGURE: *There are two leading methods for estimating the density at a point, here at the center of each square. The one shown in the top row is to start with a large volume centered on the test point and shrink it according to a function such as Vn = 1 /√n. The other method, shown in the bottom row, is to decrease the volume in a data-dependent way, for instance letting the volume enclose some number kn = √n of sample points. The sequences in both cases represent random variables that generally converge and allow the true density at the test point to be calculated.*

PARZEN WINDOWS

- Parzen window, also known as ***Parzen-Rosenblatt window,*** is a ***non-parametric method*** used for density estimation in statistics and machine learning.
- It estimates the probability density function of a random variable by placing a kernel function at each data point and summing their contributions. This technique is particularly useful when the underlying data distribution is unknown.
- The Parzen-window approach to estimating densities can be introduced by temporarily assuming that the region R_n is a d-dimensional hypercube. If h_n is the length of an edge of that hypercube, then its volume is given by:

$$V_n = h_n^d$$

We can obtain an analytic expression for k_n, the number of samples falling in the hypercube, by defining the following *window function:*

$$\varphi(\mathbf{u}) = \begin{cases} 1 & |u_j| \le 1/2; \quad j = 1, \ldots, d \\ 0 & \text{otherwise.} \end{cases}$$

- Thus, $\Phi(u)$ defines a unit hypercube centered at the origin. It follows that $\Phi((x - X_i)/h_n)$ is equal to unity if x_i falls within the hypercube of volume V_n centered at x, and is zero otherwise. The number of samples in this hypercube is therefore given by:

$$k_n = \sum_{i=1}^{n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right)$$

Estimate:

$$p_n(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{V_n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right)$$

To be more precise, if we require that:

$$\varphi(\mathbf{x}) \geq 0$$

$$\int \varphi(\mathbf{u}) d\mathbf{u} = 1$$

and if we maintain the relation $V_n = h^d_n$, then it follows at once that $p_n(x)$ also satisfies these conditions.

Effect of *window width* h_n has on $p_n(x)$. If we define the function δ(x) by

$$\delta_n(\mathbf{x}) = \frac{1}{V_n}\varphi\left(\frac{\mathbf{x}}{h_n}\right)$$

$$p_n(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n}\delta_n(\mathbf{x}-\mathbf{x}_i)$$

For any value of h_n, the distribution is normalized, that is:

$$\int \delta_n(\mathbf{x}-\mathbf{x}_i)\,d\mathbf{x} = \int \frac{1}{V_n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right) d\mathbf{x} = \int \varphi(\mathbf{u})d\mathbf{u} = 1$$

Thus, as h_n approaches zero, $\delta_n(x - x_i)$ approaches a Dirac delta function centered at x_i and $p_n(x)$ approaches a superposition of delta functions centered at the samples.

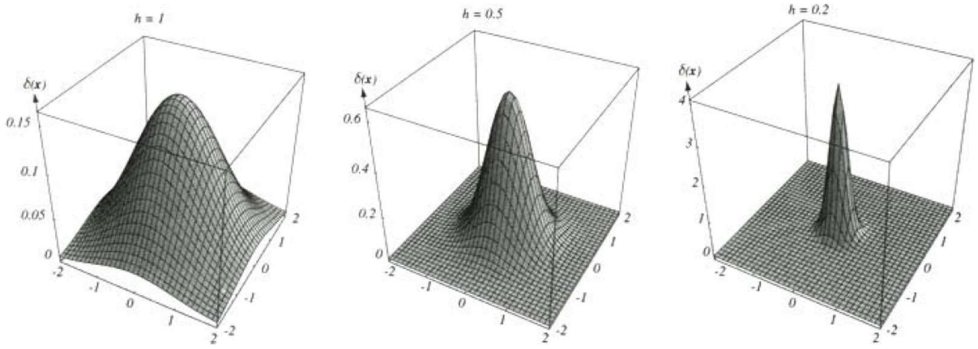

Figure: Examples of two-dimensional circularly symmetric normal Parzen windows for three different values of h. Note that because the δ (x) are normalized, different vertical scales must be used to show their structure.

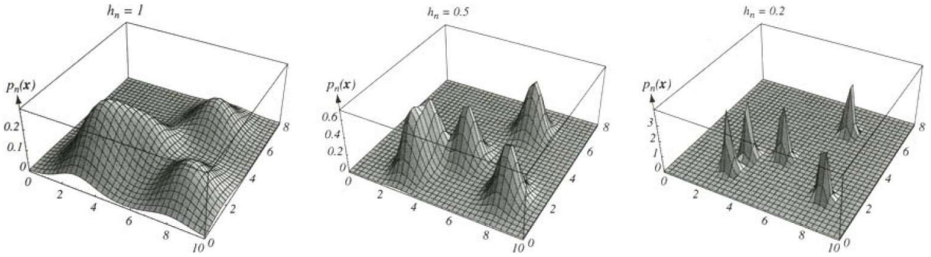

Figure: *Three Parzen-window density estimates based on the same set of five samples, using the window functions. As before, the vertical axes have been scaled to show the structure of each distribution.*

Clearly, the choice of *hn* (or V_n) has an important effect on $p_n(x)$.

- If V_n is too large, the estimate will suffer from too little resolution.

- If V_n is too small, the estimate will suffer from too much statistical variability.
- With an unlimited number of samples, it is possible to let V_n slowly approach zero as n increases and have $p_n(x)$ converge to the unknown density $p(x)$.

In discussing convergence, we must recognize that we are talking about the convergence of a sequence of random variables, because for any fixed x the value of p_n (x) depends on the random samples $x_1, \ldots, x_n$. Thus, p_n (x) has some mean $\bar{p}_n$ (x) and variance $\sigma^2(x)$. We shall say that the estimate $p_n(x)$ converges to $p(x)$ if*

$$\lim_{n\to\infty} \bar{p}_n(\mathbf{x}) = p(\mathbf{x})$$

$$\lim_{n\to\infty} \sigma_n^2(\mathbf{x}) = 0$$

Conditions assure convergence:

$$\sup_{\mathbf{u}} \varphi(\mathbf{u}) < \infty$$

$$\lim_{\|\mathbf{u}\|\to\infty} \varphi(\mathbf{u}) \prod_{i=1}^{d} u_i = 0$$

$$\lim_{n\to\infty} V_n = 0$$

$$\lim_{n\to\infty} n V_n = \infty.$$

1. Convergence of the Mean

Consider first $\bar{p}_n(x)$, the mean of $p_n(x)$. Because the samples x, are i.i.d. according to the (unknown) density $p(x)$, we have:

$$\begin{aligned}
\bar{p}_n(\mathbf{x}) &= \mathcal{E}[p_n(\mathbf{x})] \\
&= \frac{1}{n}\sum_{i=1}^{n} \mathcal{E}\left[\frac{1}{V_n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right)\right] \\
&= \int \frac{1}{V_n}\varphi\left(\frac{\mathbf{x}-\mathbf{v}}{h_n}\right) p(\mathbf{v})\, d\mathbf{v} \\
&= \int \delta_n(\mathbf{x}-\mathbf{v})\, p(\mathbf{v})\, d\mathbf{v}.
\end{aligned}$$

2. Convergence of the Variance

$$\sigma_n^2(\mathbf{x}) \le \frac{\sup(\varphi(\cdot))\ \bar{p}_n(\mathbf{x})}{n V_n}$$

3. Illustrations

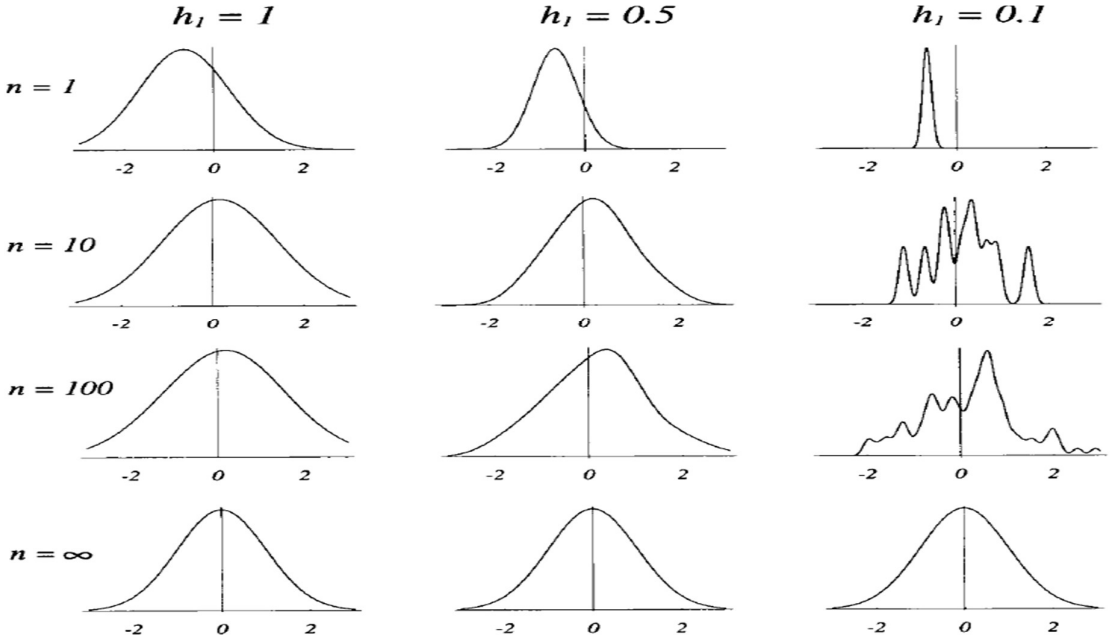

Figure: *Parzen-window estimates of a univariate normal density using different window widths and numbers of samples. The vertical axes have been scaled to best show the structure in each graph. Note particularly that the n = ∞ estimates are the same (and match the true density function), regardless of window width.*

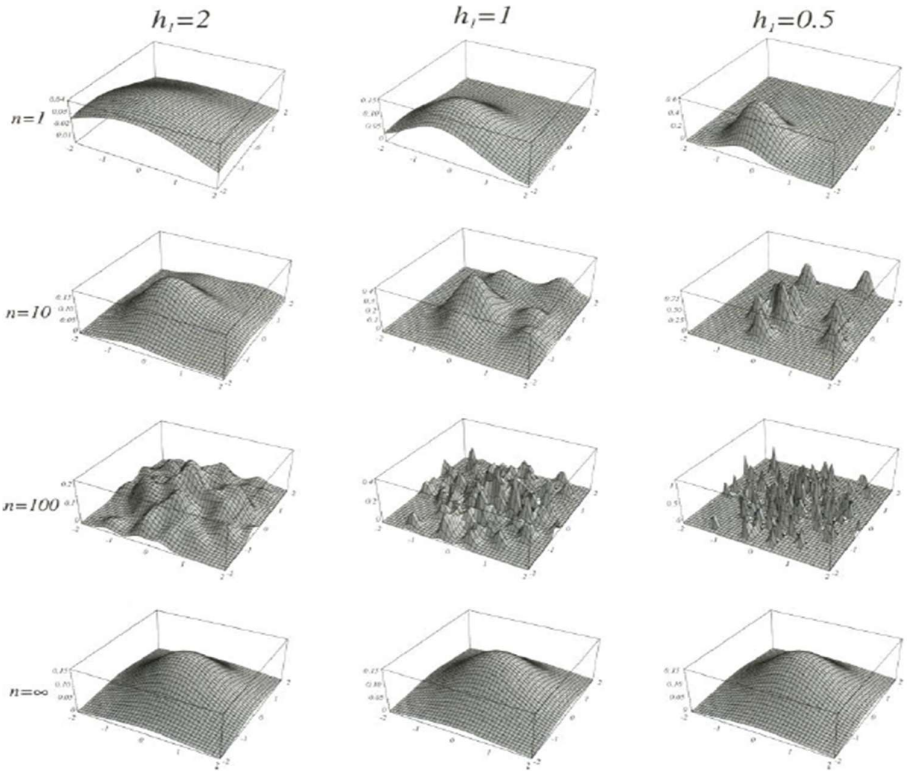

Figure: *Parzen-window estimates of a bivariate normal density using different window widths and numbers of samples. The vertical axes have been scaled to best show the structure in each*

graph. Note particularly that the n = ∞ estimates are the same (and match the true distribution), regardless of window width.

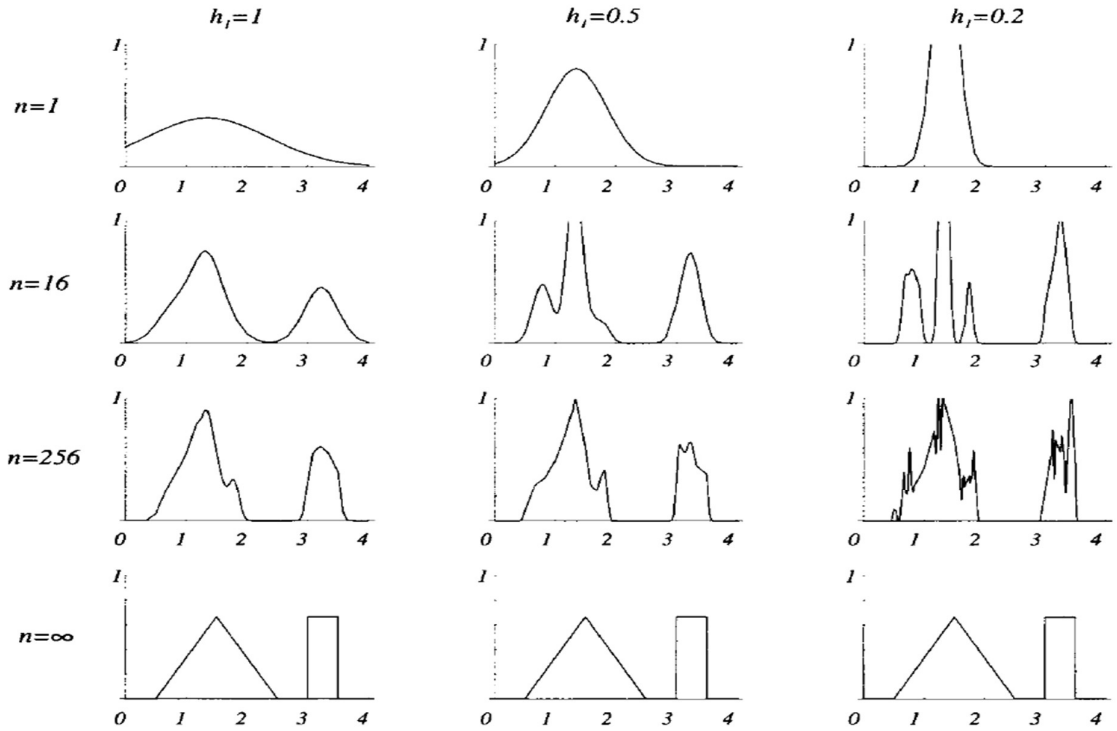

Figure: *Parzen-window estimates of a bimodal distribution using different window widths and numbers of samples. Note particularly that the n = ∞ estimates are the same (and match the true distribution), regardless of window width.*

4. Classification Example

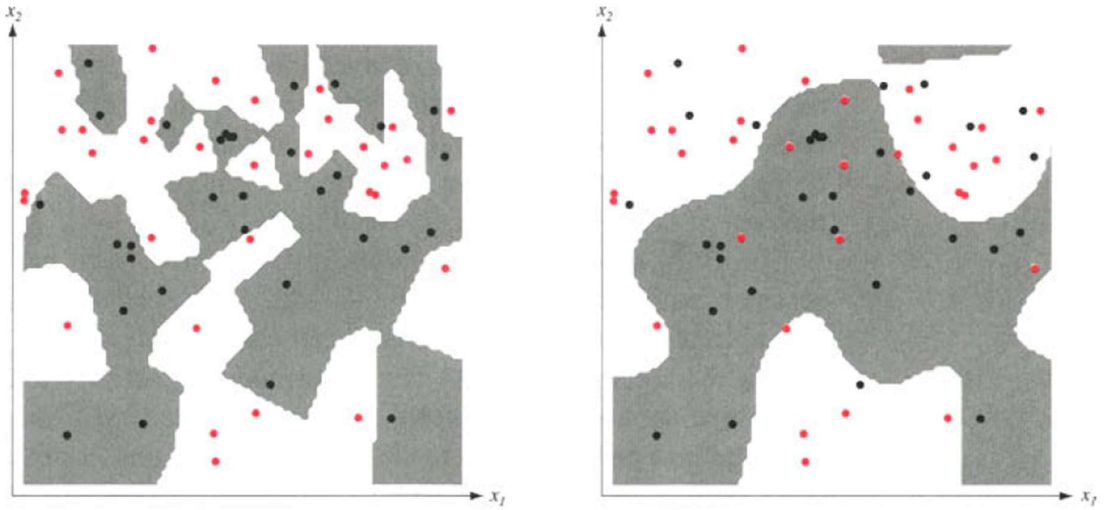

Figure: *The decision boundaries in a two-dimensional Parzen-window dichotomizer depend on the window width h. At the left a small h leads to boundaries that are more complicated than for large h on same data set, shown at the right. Apparently, for these data a small h*

would be appropriate for the upper region, while a large h would be appropriate for the lower region; no single window width is ideal overall.

5. Probabilistic Neural Networks (PNNs)

Suppose we wish to form a Parzen estimate based on *n* patterns, each of which is *d-dimensional*, randomly sampled from *c* classes. The PNN for this case consists of *d input units* comprising the input layer, where each unit is connected to each of the *n pattern units;* each pattern unit is, in turn, connected to one and only one of the *c* category units. The connections from the input to pattern units represent modifiable *weights,* which will be trained. (While these weights are merely parameters and could be represented by a vector Θ, in keeping with the established terminology in neural networks we shall use the symbol w.) Each category unit computes the sum of the pattern units connected to it.

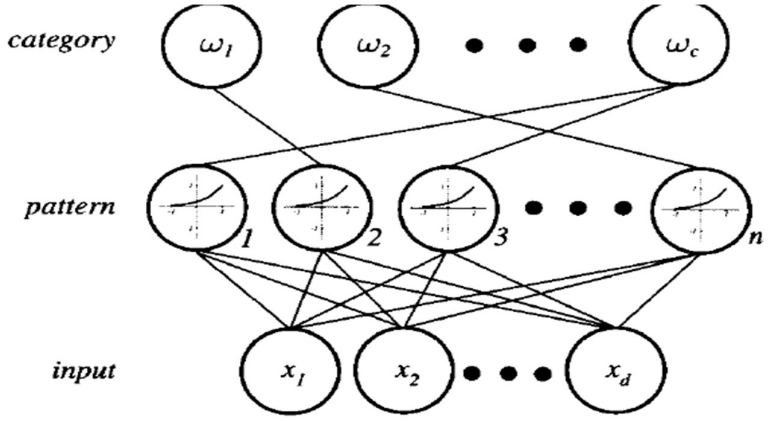

Figure: A probabilistic neural network (PNN) consists of d input units, n pattern units, and c category units. Each pattern unit forms the inner product of its weight vector and the normalized pattern vector x to form $z = w^t x$, and then it emits $\exp[(z - 1)/\sigma^2]$. Each category unit sums such contributions from the pattern unit connected to it. This ensures that the activity in each of the category units represents the Parzen-window density estimate using a circularly symmetric Gaussian window of covariance $\boldsymbol{\sigma^2 I}$, where I is the d-by-d identity matrix.

Algorithm 1. (PNN training)

1 **begin initialize** $j \leftarrow 0, n, a_{ji} \leftarrow 0$ for $j = 1, \ldots, n;\ i = 1, \ldots, c$
2 **do** $j \leftarrow j + 1$
3 $x_{jk} \leftarrow x_{jk} \Big/ \left(\sum_i^d x_{ji}^2\right)^{1/2}$ (normalize)
4 $w_{jk} \leftarrow x_{jk}$ (train)
5 **if** $\mathbf{x}_j \in \omega_i$ **then** $a_{ji} \leftarrow 1$
6 **until** $j = n$
7 **end**

NET ACTIVATION:

$$net_k = \mathbf{w}_k^t \mathbf{x}$$

Algorithm 2. (PNN Classification)

1 **begin initialize** $k \leftarrow 0$, $\mathbf{x} \leftarrow$ test pattern
2 **do** $k \leftarrow k + 1$
3 $net_k \leftarrow \mathbf{w}_k^t \mathbf{x}$
4 **if** $a_{ki} = 1$ **then** $g_i \leftarrow g_i + \exp[(net_k - 1)/\sigma^2]$
5 **until** $k = n$
6 **return** $class \leftarrow \arg\max_i g_i(\mathbf{x})$
7 **end**

6. Choosing the Window Function

- If V_1 is too small, most of the volumes will be empty, and the estimate p_n (x) will be very erratic.
- If V_1 is too large, important spatial variations in *p(x)* may be lost due to averaging over the cell volume.

K_n-NEAREST-NEIGHBOR ESTIMATION

- A potential remedy for the problem of the unknown "best" window function is to let the cell volume be a function of the *training data,* rather than some arbitrary function of the overall number of samples. For example, to estimate *p(x)* from *n* training samples or *prototypes* we can center a cell about x and let it grow until it captures k_n samples, where k_n is some specified function of *n.* These samples are the k_n *nearest-neighbors* of x. If the density is high near x, the cell will be relatively small, which leads to good resolution. If the density is low, it is true that the cell will grow large, but it will stop soon after it enters regions of higher density. In either case, if we take:

$$p_n(\mathbf{x}) = \frac{k_n/n}{V_n}$$

  we want k_n to go to infinity as *n* goes to infinity, since this assures us that k_n */n* will be a good estimate of the probability that a point will fall in the cell of volume V_n.
- However, we also want k_n to grow sufficiently slowly that the size of the cell needed to capture *kn* training samples will shrink to zero.
- The ratio k_n/n must go to zero.
- The conditions $\lim_{n\to\infty} k_n = \infty$ and $\lim_{n\to\infty} k_n/n = 0$ are necessary and sufficient for p_n *(x)* to converge to *p(x)* in probability at all points where *p(x)* is continuous.

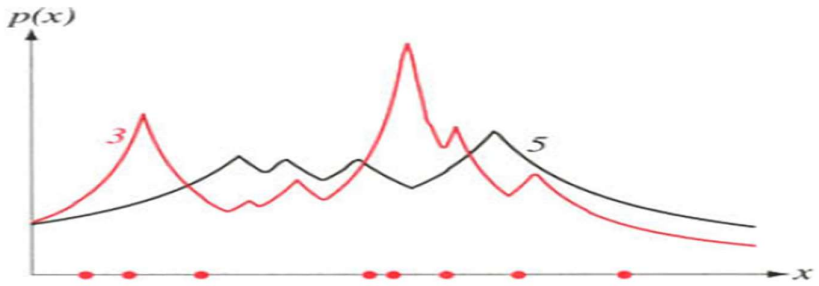

Figure: Eight points in one dimension and the k-nearest-neighbor density estimates, for k = 3 and 5. Note especially that the discontinuities in the slopes in the estimates generally lie away from the positions of the prototype points.

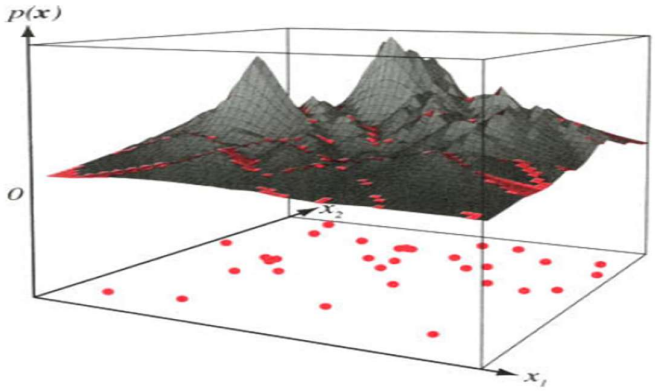

Figure: The k-nearest-neighbor estimate of a two-dimensional density for k = 5. Notice how such a finite n estimate can be quite "jagged," and notice that discontinuities in the slopes generally occur along lines away from the positions of the points themselves.

k_n-Nearest-Neighbor and Parzen-Window Estimation:

- With $n = 1$ and $k_n = \sqrt{n} = 1$, the estimate becomes:

$$p_n(x) = \frac{1}{2|x - x_1|}$$

- Fact that $p_n(x)$ never plunges to zero just because no samples fall within some arbitrary cell or window.
- As with the Parzen-window approach, we could obtain a family of estimates by taking $k_n = k_1 \sqrt{n}$ and choosing different values for k_1.
- For classification, one popular method is to adjust the window width until the classifier has the lowest error on a separate set of samples, also drawn from the target distributions.

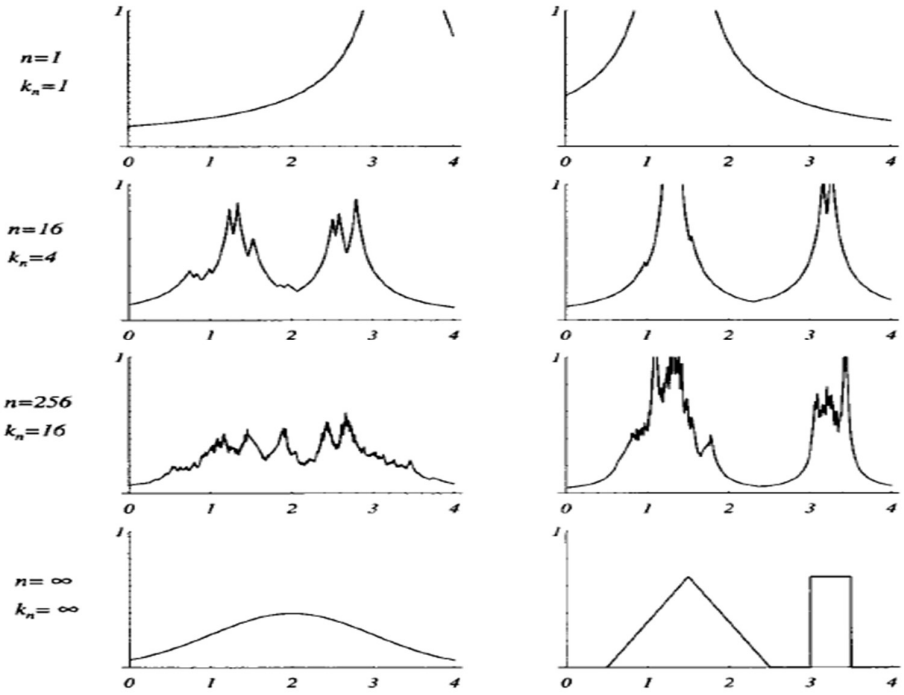

Figure: Several k-nearest-neighbor estimates of two unidimensional densities: a Gaussian and a bimodal distribution. Notice how the finite n estimates can be quite "spiky."

Estimation of *a Posteriori* Probabilities

To estimate the *a posteriori* probabilities $P(\omega_i| x)$ from a set of n labeled samples by using the samples to estimate the densities involved. Suppose that we place a cell of volume V around x and capture k samples, k_i of which turn out to be labeled ω_i. Then the obvious estimate for the joint probability $p(x, \omega_i)$ is:

$$p_n(\mathbf{x}, \omega_i) = \frac{k_i/n}{V}$$

A reasonable estimate for $P(\omega_i)$ |x) is:

$$P_n(\omega_i|\mathbf{x}) = \frac{p_n(\mathbf{x}, \omega_i)}{\sum_{j=1}^{c} p_n(\mathbf{x}, \omega_j)} = \frac{k_i}{k}.$$

- The estimate of the *a posteriori* probability that ω_i, is the state of nature is merely the fraction of the samples within the cell that are labeled ω_i.
- For minimum error rate we select the category most frequently represented within the cell. If there are enough samples and if the cell is sufficiently small, it can be shown that this will yield performance approaching the best possible.
- When it comes to choosing the size of the cell, it is clear that we can use either the Parzen-window approach or the k_n -nearest-neighbor approach.
    - In the first case, V_n would be some specified function of n, such as $V_n = 1/\sqrt{n}$.
    - In the second case, V_n would be expanded until some specified number of samples were captured, such as $k = \sqrt{n}$.
    - In either case, as n goes to infinity an infinite number of samples will fall within the infinitely small cell.

- The fact that the cell volume could become arbitrarily small and yet contain an arbitrarily large number of samples would allow us to learn the unknown probabilities with virtual certainty and thus eventually obtain optimum performance.

THE NEAREST-NEIGHBOR RULE

We begin by letting $D^n = \{x_1,..., x_n\}$ denote a set of *n* labeled prototypes and letting $x' \in D^n$ be the prototype nearest to a test point x.
Then the *nearest-neighbor rule* for classifying x is to assign it the label associated with x'.
The nearest-neighbor rule is a suboptimal procedure; its use will usually lead to an error rate greater than
the minimum possible, the Bayes rate.
With an unlimited number of prototypes the error rate is never worse than twice the Bayes rate.
To begin with, note that the label Θ' associated with the nearest neighbor is a random variable, and the probability that $\Theta' = \omega_i$ is merely the *a posteriori* probability $P(\omega_i|x')$.
When the number of samples is very large, it is reasonable to assume that x' is sufficiently close to x that

$$P(\omega_i|x') = (\omega_i|x).$$

If we define ω_m (x) by:

$$P(\omega_m|\mathbf{x}) = \max_i P(\omega_i|\mathbf{x})$$

then the Bayes decision rule always selects ω_m. This rule allows us to partition the feature space into cells consisting of all points closer to a given training point x' than to any other training points. All points in such a cell are thus labeled by the category of the training point—a so-called *Voronoi tesselation* of the space.

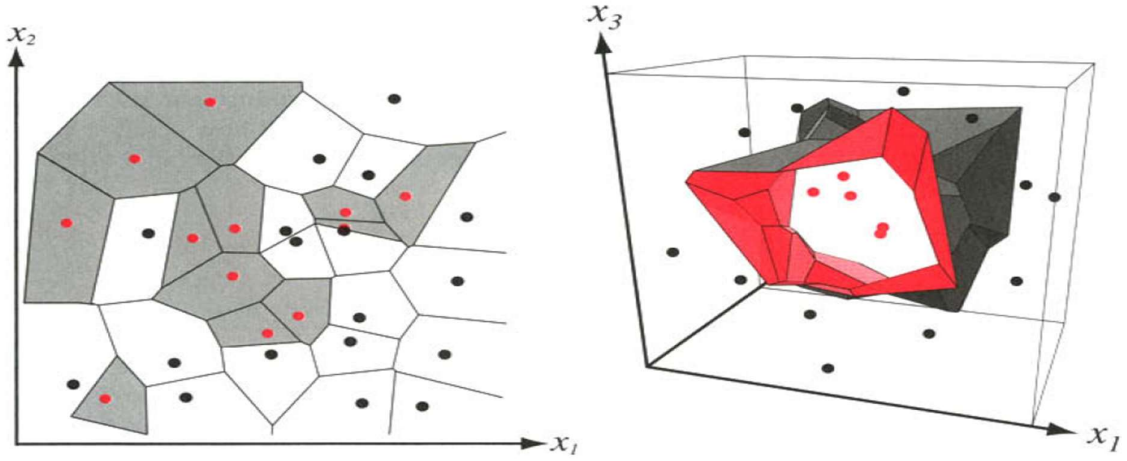

Figure: In two dimensions, the nearest-neighbor algorithm leads to a partitioning of the input space into Voronoi cells, each labeled by the category of the training point it contains. In three dimensions, the cells are three-dimensional, and the decision boundary resembles the surface of a crystal.

When $P(\omega_m|\ x)$ is close to unity, the nearest-neighbor selection is almost always the same as the Bayes selection. That is, when the minimum probability of error is small, the nearest-neighbor probability of error is also small. When $P(\omega_m|x)$ is close to 1/c, so that all classes are essentially equally likely, the selections made by the nearest-neighbor rule and the Bayes decision rule are rarely the same, but the probability of error is approximately 1-1/c for both.

The unconditional average probability of error will then be found by averaging $P(e|x)$ over all x:

$$P(e) = \int P(e|\mathbf{x})p(\mathbf{x})\,d\mathbf{x}.$$

The Bayes decision rule minimizes $P(e)$ by minimizing $P(e|x)$ for every x. If we let $P^*(e|x)$ be the minimum
possible value of $P(e|x)$, and P^* be the minimum possible value of $P(e)$, then

$$P^*(e|\mathbf{x}) = 1 - P(\omega_m|\mathbf{x})$$

$$P^* = \int P^*(e|\mathbf{x})p(\mathbf{x})\,d\mathbf{x}.$$

1. Convergence of the Nearest Neighbor

Evaluate the average probability of error for the nearest-neighbor rule. In particular, if $P_n(e)$ is the n-sample error rate, and if:

$$P = \lim_{n\to\infty} P_n(e)$$

Under these conditions, the probability that any sample falls within a hypersphere S centered about x is some positive number Ps :

$$P_s = \int_{\mathbf{x}'\in\mathcal{S}} p(\mathbf{x}')\,d\mathbf{x}'.$$

Thus, the probability that all n of the independently drawn samples fall outside this hypersphere is $(1 - Ps)^n$, which approaches zero as n goes to infinity.

2. Error Rate for the Nearest-Neighbor Rule

Denote the nearest neighbor by x_n'. When we say that we have n independently drawn labeled samples, we are talking about n pairs of random variables (x_1, Θ_1), (x_2, Θ_2),..., (x_n, Θ_n), where Θ_j may be any of the c states of nature $\omega_1 \ldots \omega_c$. We assume that these pairs were generated by selecting a state of nature ω_j for Θ_j with probability $P(\omega_j)$ and then selecting an x_j according to the probability law $p(x|\omega_j)$, with each pair being selected independently.
Suppose that during classification, nature selects a pair (x, Θ), and also suppose that x'_n, labeled Θ'_n, is the training sample nearest x. Because the state of nature when x'_n was drawn is independent of the state of nature when x is drawn, we have:

$$P(\theta, \theta'_n|\mathbf{x}, \mathbf{x}'_n) = P(\theta|\mathbf{x})P(\theta'_n|\mathbf{x}'_n).$$

Now if we use the nearest-neighbor decision rule, we commit an error whenever $\Theta = \Theta'_n$. Thus, the conditional probability of error $P_n(e|x, x'_n)$ is given by

$$P_n(e|\mathbf{x}, \mathbf{x}'_n) = 1 - \sum_{i=1}^{c} P(\theta = \omega_i, \theta'_n = \omega_i|\mathbf{x}, \mathbf{x}'_n)$$

$$= 1 - \sum_{i=1}^{c} P(\omega_i|\mathbf{x})P(\omega_i|\mathbf{x}'_n).$$

3. Error Bounds

The problem of establishing a tight upper bound is more interesting.

$$P^*(e|\mathbf{x}) = 1 - P(\omega_m|\mathbf{x}),$$

$$P(\omega_i|\mathbf{x}) = \begin{cases} \dfrac{P^*(e|\mathbf{x})}{c-1} & i \neq m \\ 1 - P^*(e|\mathbf{x}) & i = m \end{cases}$$

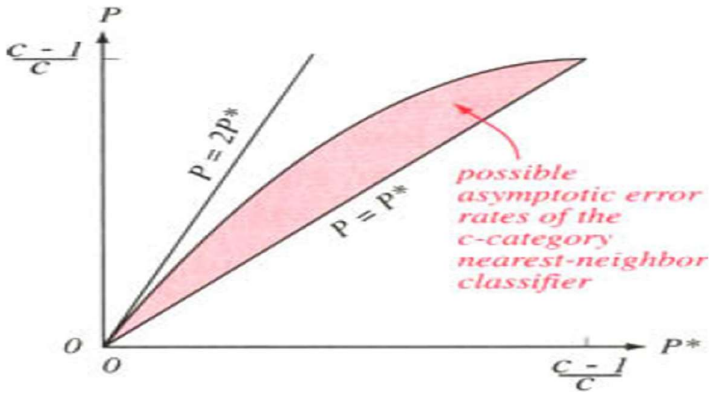

Figure: Bounds on the nearest-neighbor error rate P in a c-category problem given infinite training data, where P is the Bayes error. At low error rates, the nearest-neighbor error rate is bounded above by twice the Bayes rate.*

4. The k-Nearest-Neighbor Rule

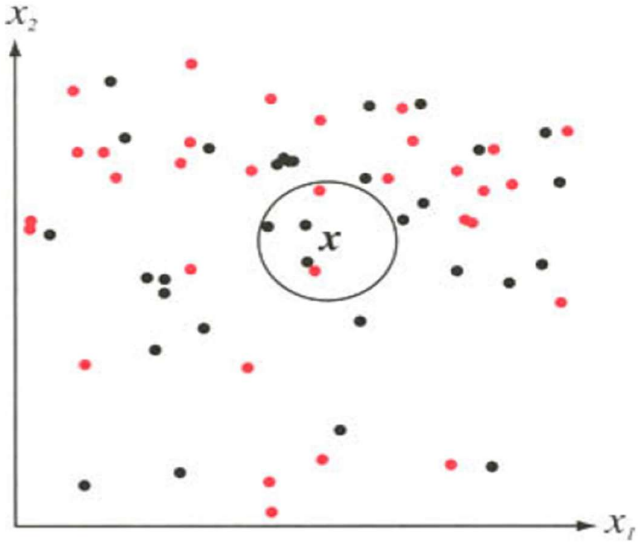

Figure: The k-nearest-neighbor query starts at the test point x and grows a spherical region until it encloses k training samples, and it labels the test point by a majority vote of these samples. In this k = 5 case, the test point x would be labeled the category of the black points.

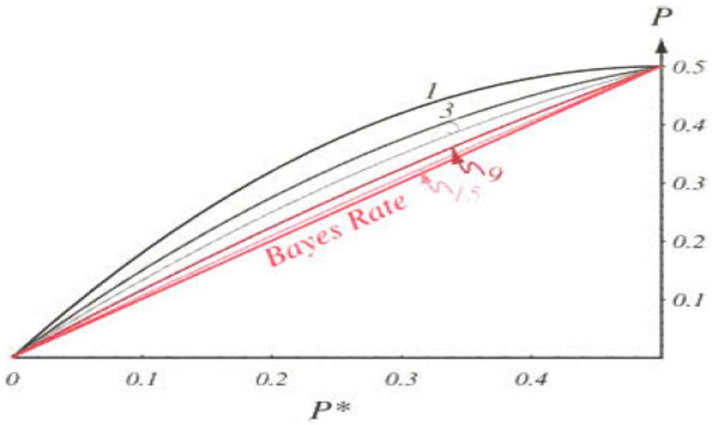

Figure: The error rate for the k-nearest-neighbor rule for a two-category problem is bounded by $C_k(P^*)$. Each curve is labeled by k; when $k = \infty$, the estimated probabilities match the true probabilities and thus the error rate is equal to the Bayes rate, that is, $P = P^*$.

Large value of *k* to obtain a reliable estimate. We want all of the *k* nearest neighbors x' to be very near x to be sure that $P(\omega_i |x')$ is approximately the same as $P(\omega_i |x)$. This forces us to choose a compromise *k* that is a small fraction of the number of samples. It is only in the limit as *n* goes to infinity that we can be assured of the nearly optimal behavior of the k-nearest-neighbor rule.

5. Computational Complexity of the k-Nearest-Neighbor Rule

Suppose we have *n* labeled training samples in *d* dimensions and we seek the (single) closest to a test point x *(k* = 1). In the most naive approach we inspect each stored point in turn, calculate its Euclidean distance to x, retaining the identity only of the current closest one. Each distance calculation is *0(d),* and thus this search is O(dn). But, in parallel implementation is 0(1) in time and *0(n)* in space.

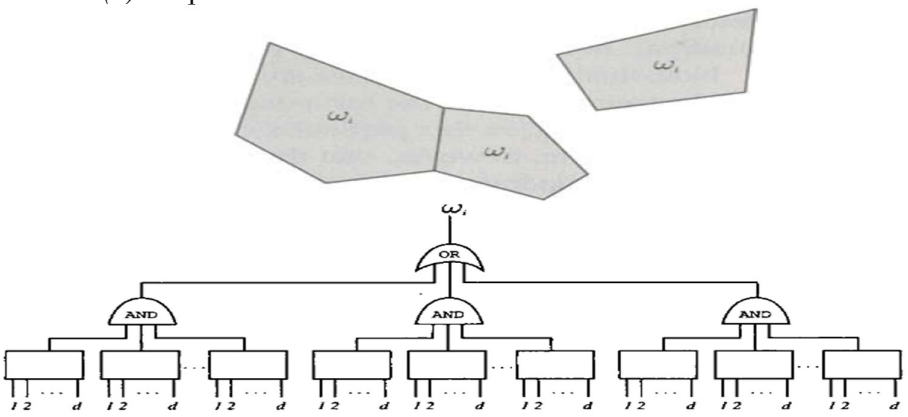

Figure: A parallel nearest-neighbor circuit can perform search in constant—that is, 0(1)—time. The d-dimensional test pattern x is presented to each box, which calculates which side of a cell's face x lies on. If it is on the "close" side of every face of a cell, it lies in the Voronoi cell of the stored pattern, and receives its label. In the case shown, each of the three ***AND*** *gates corresponds to a single Voronoi cell.*

Algorithm 3. (Nearest-Neighbor Editing)

```
1 begin initialize j ← 0, 𝒟 ← data set, n ← # prototypes
2         construct the full Voronoi diagram of 𝒟
3         do j ← j + 1;  for each prototype x'_j
4            find the Voronoi neighbors of x'_j
5            if any neighbor is not from the same class as x'_j, then mark x'_j
6         until j = n
7   discard all points that are not marked
8   construct the Voronoi diagram of the remaining (marked) prototypes
9 end
```

The complexity of this editing algorithm is $O(d^3 n^{\lfloor d/2 \rfloor} \ln n)$, where the "floor" operation ([.]) implies $\lfloor d/2 \rfloor = k$ if d is even and $2k - 1$ if d is odd.

METRICS AND NEAREST-NEIGHBOR CLASSIFICATION

A metric $D(\cdot)$ is merely a function that gives a generalized scalar distance between two argument patterns.

1. Properties of Metrics

A metric must have four properties: For all vectors a, b, and c, these properties are as follows:

nonnegativity: $D(\mathbf{a}, \mathbf{b}) \geq 0$

reflexivity: $D(\mathbf{a}, \mathbf{b}) = 0$ if and only if $\mathbf{a} = \mathbf{b}$

symmetry: $D(\mathbf{a}, \mathbf{b}) = D(\mathbf{b}, \mathbf{a})$

triangle inequality: $D(\mathbf{a}, \mathbf{b}) + D(\mathbf{b}, \mathbf{c}) \geq D(\mathbf{a}, \mathbf{c})$

A. The **Euclidean formula** for distance in d dimensions:

$$D(\mathbf{a}, \mathbf{b}) = \left(\sum_{k=1}^{d} (a_k - b_k)^2 \right)^{1/2}$$

possesses the properties of metric.

B. One general class of metrics for d-dimensional patterns is the **Minkowski metric**:

$$L_k(\mathbf{a}, \mathbf{b}) = \left(\sum_{i=1}^{d} |a_i - b_i|^k \right)^{1/k}$$

The L_1 norm is sometimes called the *Manhattan* or *city block* distance, the shortest path between a and b, each segment of which is parallel to a coordinate axis.

C. The ***Tanimoto metric*** finds most use in taxonomy, where the distance between two sets is defined as:

$$D_{Tanimoto}(S_1, S_2) = \frac{n_1 + n_2 - 2n_{12}}{n_1 + n_2 - n_{12}}$$

where n_1 and n_2 are the number of elements in sets S_1 and $S2$, respectively, and n_{12} is the number that is in both sets. The Tanimoto metric finds greatest use for problems in which two patterns or features—the elements in the set—are either the same or different, and there is no natural notion of graded similarity.

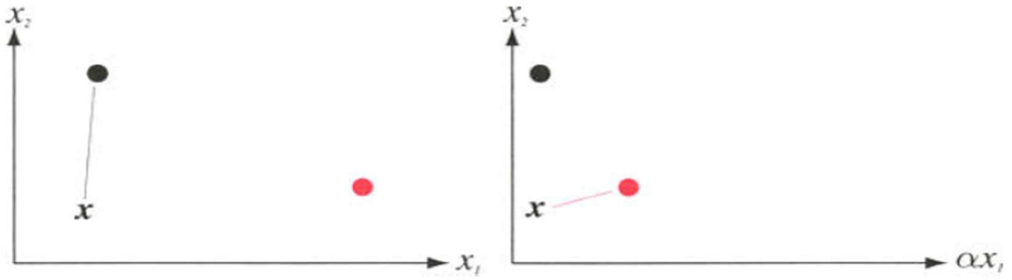

Figure: Scaling the coordinates of a feature space can change the distance relationships computed by the Euclidean metric. Here we see how such scaling can change the behavior of a nearest-neighbor classifer. Consider the test point x and its nearest neighbor. In the original space (left), the black prototype is closest. In the figure at the right, the X_1 axis has been rescaled by a factor a = 1 / 3 ; now the nearest prototype is the red one. If there is a large disparity in the ranges of the full data in each dimension, a common procedure is to rescale all the data to equalize such ranges, and this is equivalent to changing the metric in the original space.

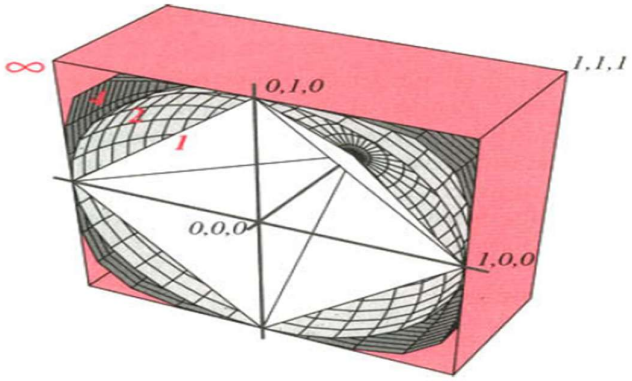

Figure: Each colored surface consists of points a distance 1.0 from the origin, measured using different values for k in the Minkowski metric (k is printed in red). Thus the white surfaces correspond to the L_1 norm (Manhattan distance), the light gray sphere corresponds to the L2 norm (Euclidean distance), the dark gray ones correspond to the L4 norm, and the pink box corresponds to the L_∞ norm.

2. Tangent Distance

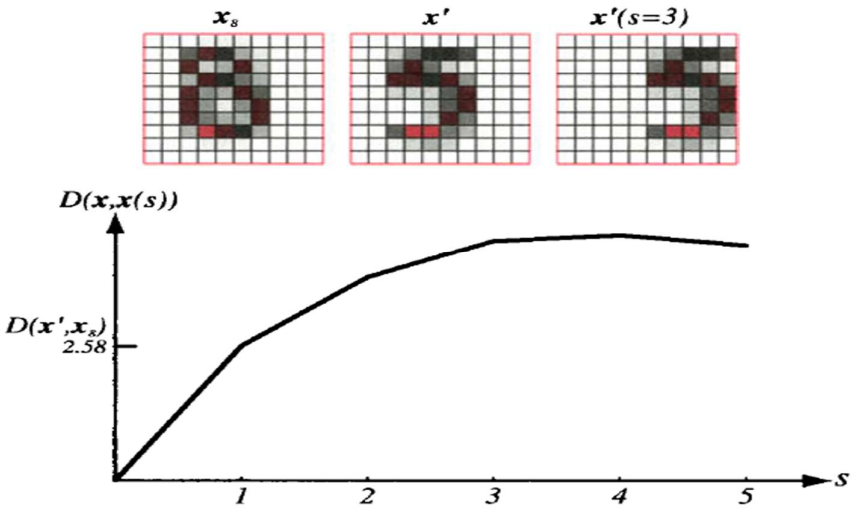

Figure: The uncritical use of Euclidean metric cannot address the problem of translation invariance. Pattern x' represents a handwritten 5, and x'(s = 3) represents the same shape but shifted three pixels to the right. The Euclidean distance D(x', x'(s = 3)) is much larger than D(x', x8), where x8 represents the handwritten 8. Nearest-neighbor classification based on the Euclidean distance in this way leads to very large errors. Instead, we seek a distance measure that would be insensitive to such translations, or indeed other known invariances, such as scale or rotation.

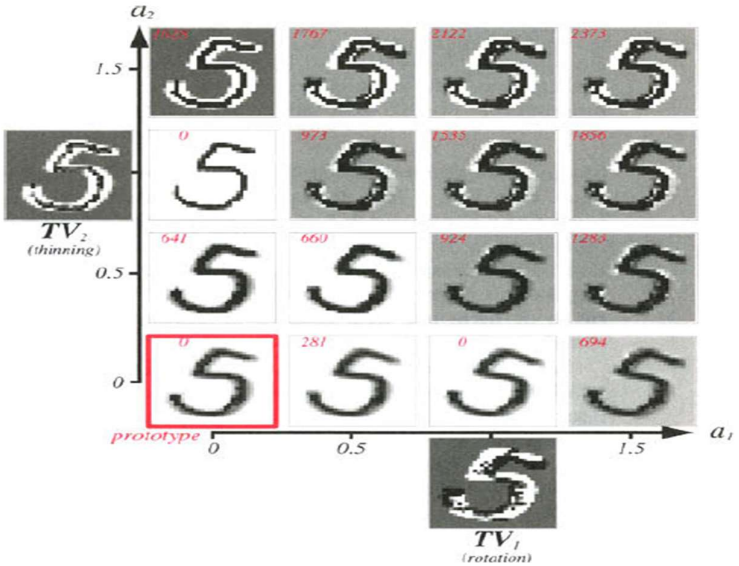

*Figure: The pixel image of the handwritten 5 prototype at the lower left was subjected to two transformations, rotation, and line thinning, to obtain the tangent vectors **TV1** and TV2; images corresponding to these tangent vectors are shown outside the axes. Each of the 16 images within the axes represents the prototype plus linear combination of the two tangent vectors with coefficients a1 and a2. The small red number in each image is the Euclidean distance between the tangent approximation and the image generated by the unapproximated transformations. Of course, this Euclidean distance is 0 for the prototype and for the cases a1 = 1, a_2 = 0 and a_1 = 0, a2 = 1. (The patterns generated with a_1 + a2 > 1 have a gray background because of automatic grayscale conversion of images with negative pixel values.)*

The general approach in tangent distance classifiers is to use a novel measure of distance and a linear *approximation* to the arbitrary transforms. Suppose we believe there are *r* transformations applicable to our problem, such as horizontal translation, vertical translation, shear, rotation, scale, and line thinning.
During construction of the classifier we take each stored prototype x' and perform each of the transformations
F_i (x ' ; α_i) on it. Thus F_i (x ' ; α_i) could represent the image described by x', rotated by a small angle α_i. We then construct a *tangent vector* TV_i for each transformation:

$$\mathbf{TV}_i = \mathcal{F}_i(\mathbf{x}';\ \alpha_i) - \mathbf{x}'$$

For each prototype x' an r x *d* matrix T, consisting of the tangent vectors at x'.
To compute the tangent distance from a test point x to a particular stored prototype x'. Formally, given a matrix T consisting of the *r* tangent vectors at x', the tangent distance from x' to x is:

$$D_{tan}(\mathbf{x}', \mathbf{x}) = \min_{\mathbf{a}}[\|(\mathbf{x}' + \mathbf{Ta}) - \mathbf{x}\|],$$

that is, the Euclidean distance from x to the tangent space of x'. This is also called "one-sided" tangent distance, because only one pattern, x', is transformed.

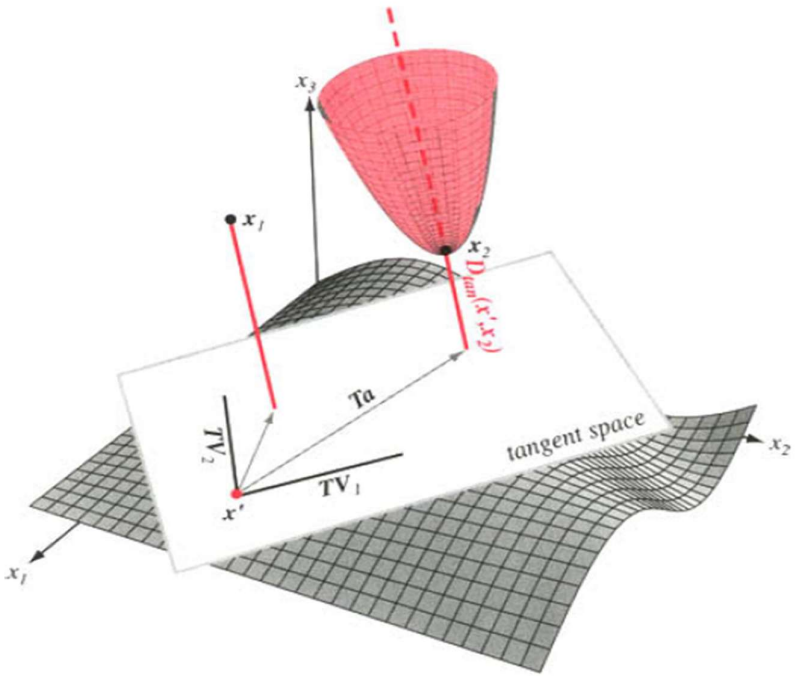

Figure: A stored prototype x', if transformed by combinations of two basic transformations, would fall somewhere on a complicated curved surface in the full d-dimensional space (gray). The tangent space at x' is an r-dimensional Euclidean space, spanned by the tangent vectors (here TV_1 and TV_2). The tangent distance D_{tan}(x', x) is the smallest Euclidean distance from x to the tangent space of x', shown in the solid red lines for two points, X_1 and x2. Thus although the Euclidean distance from x' to Xi is less than that to x2, for the tangent distance the situation is reversed. The Euclidean distance from x2 to the tangent space of x' is a quadratic function of the parameter vector a, as shown by the pink paraboloid. Thus simple gradient descent methods can find the optimal vector a and hence the tangent distance D_{tan}(x', x2).

FUZZY CLASSIFICATION

The problem domain where we seek to build a classifier. For instance, an adult salmon is oblong and light in color, while a sea bass is stouter and dark. The approach taken *fuzzy classification* is to create so-called "fuzzy category memberships functions," which convert an objectively measurable parameter into a subjective "category memberships," which are then used for classification.

For instance, if we consider the feature value of lightness, we might split this into five "categories"—dark, medium-dark, medium, medium-light and light.

To convert an objective measurement in several features into a category decision about the fish, and for this we need a *merging* or *conjunction rule*—a way to take the "category memberships" (e.g., lightness and shape) and yield a number to be used for making the final decision.

Suppose the designer feels that the final category based on lightness and shape can be described as medium-light *and* oblong. While the heuristic category membership function, $\mu(.)$, converts the objective measurements to two "category memberships," we now need a *conjunction rule* to transform the component "membership values"
into a discriminant function.

$$\mu_x \cdot \mu_y$$

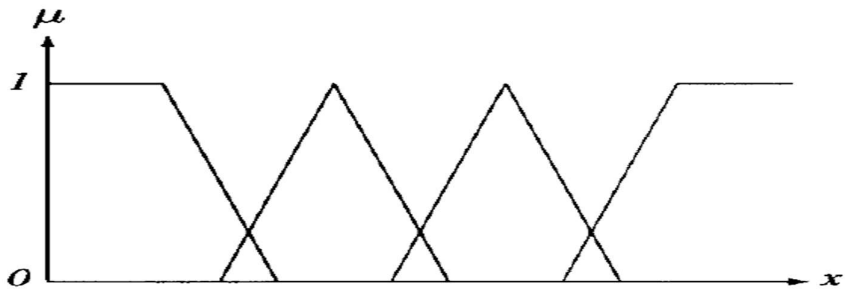

Figure: "Category membership" functions, derived from the designer's prior knowledge, together with a conjunction rule lead to discriminants. In this figure, x might represent an objectively measurable value such as the reflectivity of a fish's skin. The designer believes there are four relevant ranges, which might be called dark, medium-dark, medium-light, and light. The categories for the feature, of course, are not the same as the true categories or classes for the patterns.

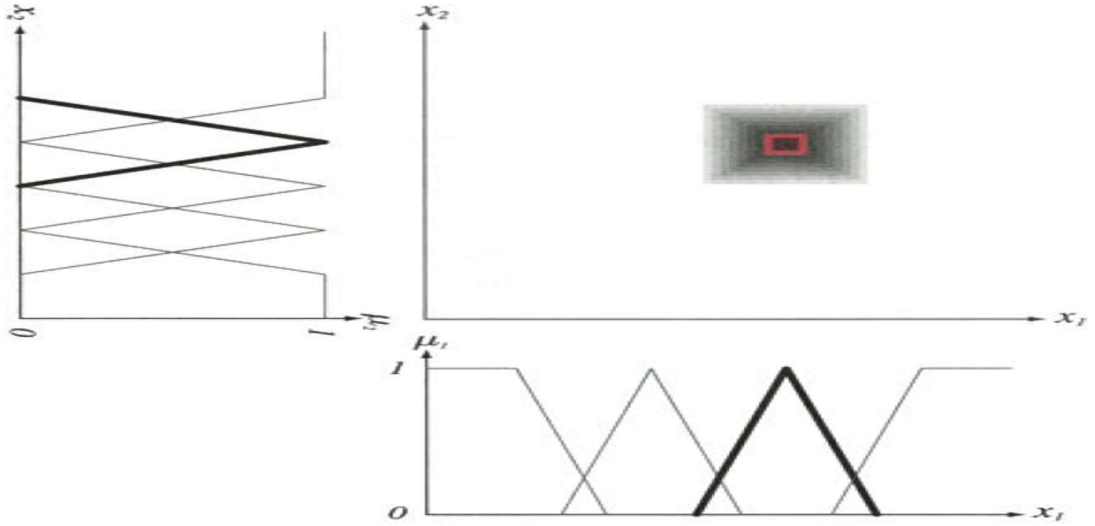

Figure: "Category membership" functions and a conjunction rule based on the designer's prior knowledge lead to discriminant functions. Here X_1 and x2 are objectively measurable feature

values. The designer believes that a particular class can be described as the conjunction of two "category memberships," here shown bold. Here the conjunction rule is used to give the discriminant function. The resulting discriminant function for the final category is indicated by the grayscale in the middle: the greater the discriminant, the darker. The designer constructs discriminant functions for other categories in a similar way (possibly also using disjunctions or other logical relations). During classification, the maximum discriminant function is chosen.

Let us assume that it is possible to compute a meaningful degree of belief about memberships in categories a, b or c given data d by mathematical functions (not necessarily probabilities) of the form P(a\d), P(b\d), and P(c\d). One very reasonable set of required characteristics is provided by Cox's axioms (also called Cox-Jaynes axioms):

1. If $P(a\backslash d) > P(b\backslash d)$ and $P(b\backslash d) > P(c\backslash d)$ then $P(a\backslash d) > P(c\backslash d)$. That is, degrees of belief have a natural ordering, given by real numbers.
2. P(not $a\backslash d) = F_1[P(a\backslash d)]$. That is, the degree of belief that a proposition is *not* the case is some function of the degree of belief that it *is* the case. Note that such degrees of belief are graded values.
3. $P(a,b\backslash d) = F2[P(a\backslash d), P(b\backslash a,d)]$.

These three axioms determine how to calculate degrees of belief—that is, how to do logical inference. We can choose the scaling such that the smallest value of any proposition is 0 and the largest is 1.0. In that case, it can be shown that the function $F_1\ (a)$ equals 1- *a,* and the function $F_2\ (a, b)$ equals *a* x *b.*

Limitations:

- Fuzzy methods are cumbersome to use in high dimensions or on complex problems or in problems with dozens or hundreds of features.
- The amount of information the designer can be expected to bring to a problem is quite limited—the number, positions, and widths of "category memberships."
- Because of their lack of normalization, pure fuzzy methods are poorly suited to problems in which there is a changing cost matrix.
- Pure fuzzy methods do not make use of training data. When such pure fuzzy methods (as outlined above) have unacceptable performance, it has been traditional to try to graft on adaptive (e.g., "neuro-fuzzy") methods.

REDUCED COULOMB ENERGY NETWORKS

- The Parzen-window method uses a fixed window throughout the feature space. However, in some regions a small window width would be appropriate, while elsewhere a large width would be appropriate. The k-nearest-neighbor method addressed this problem by adjusting the region based on the density of the points.
- Informally speaking, an approach that is intermediate between these two is to adjust the size of the window during training according to the distance to the nearest point of a *different* category. Such region adjustment can be implemented in a neural network.
- The *reduced Coulomb energy* or RCE network has the same topology as a probabilistic neural network. The method got its name because some of its equations resemble those in electrostatics describing the energy associated with a collection of charged particles. In an RCE network, each pattern unit has an adjustable parameter that corresponds to the radius of a d-dimensional sphere in the input space. During training, each radius is

adjusted so that each pattern unit covers a region as large as possible without containing a training point from another category.

Training: If the radius of a pattern unit becomes too small (i.e., less than some threshold λ_{min}), then it indicates that different categories are highly overlapping. In that case, the pattern unit is called a "probabilistic" unit and is marked accordingly.

Algorithm 4. (RCE Training)

```
1 begin initialize j ← 0, n ← # patterns, ϵ ← small param, λ_m ← max radius
2       do j ← j + 1
3          w_ij ← x_i        (train weight)
4          x̂ ← arg min_{x'∉ω_k} D(x, x')   (find nearest point not in ω_i)
5          λ_j ← min[max[D(x̂, x'), ϵ], λ_m]   (set radius)
6          if x ∈ ω_k then a_jk ← 1
7       until j = n
8 end
```

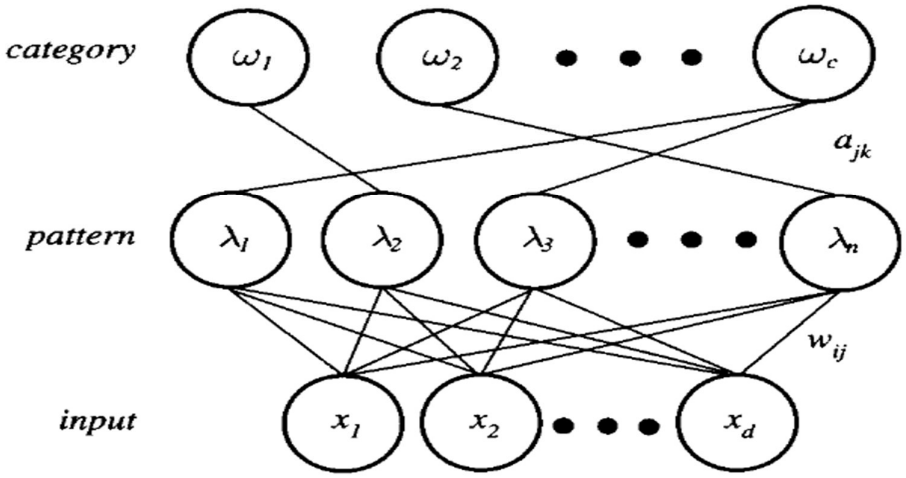

Figure: An RCE network is topologically equivalent to the PNN diagram. During training, normalized weights are adjusted to have the same values as the normalized pattern presented, just as in a PNN. In this way, distances can be calculated by an inner product. Pattern units in an RCE network also have a modifiable threshold corresponding to a "radius" X. During training, each threshold is adjusted so that its radius is as large as possible without containing training patterns from a different category.

Classification:

- During classification, a normalized test point is classified by the associated label obtained through training. Any region that is overlapped is considered ambiguous.
- Such ambiguous regions can be useful, since the teacher can be queried as to the identity of points in that region. If we continue to let λj be the radius around stored prototype x'_j- and now let D_t be the set of stored prototypes in whose hyperspheres the normalized test point x lies, then our classification algorithm is written as follows:

Algorithm 5. (RCE Classification)

```
1 begin initialize j ← 0, k ← 0, x ← test pattern, D_t ← {}
2         do j ← j + 1
3            if D(x, x'_j) < λ_j then D_t ← D_t ∪ x'_j
4         until j = n
5            if label of all x'_j ∈ D_t is the same then return label of all x_k ∈ D_t
6                                                 else return "ambiguous" label
7 end
```

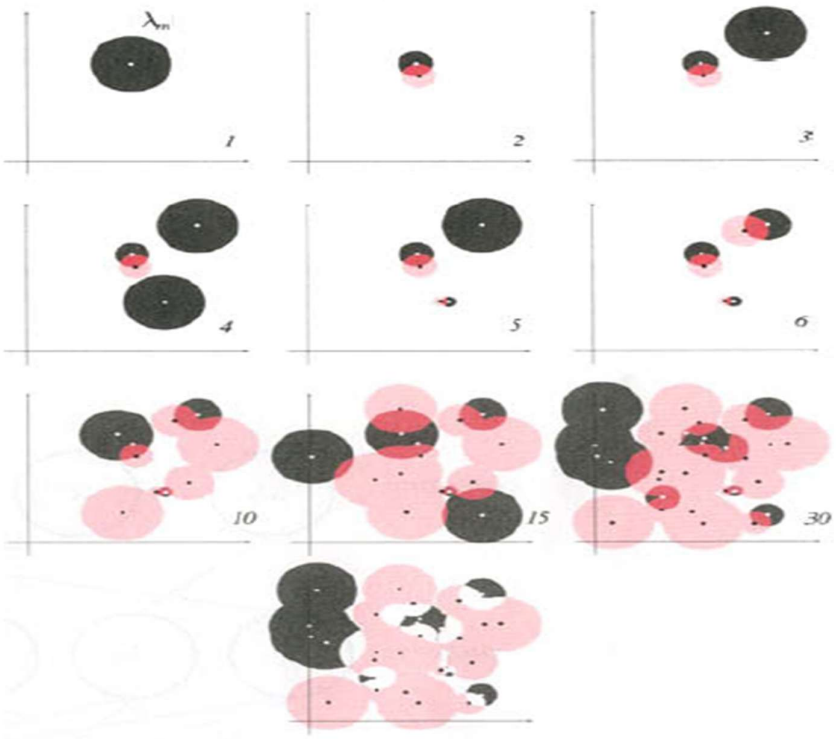

Figure: During training of an RCE network, each pattern has a parameter-equivalent to a radius in the d-dimensional space—that is adjusted to be as large as possible without enclosing any points from a different category (up to a maximum λ_m). As new patterns are presented, each such radius is decreased so that no sphere encloses a pattern of a different category. In this way, each sphere can enclose only patterns having the same category label. In this figure, the regions corresponding to one category are pink, and the other category are gray. Ambiguous regions (those enclosed by spheres of both categories) are shown in dark red. The number of points is shown in each component figure. The figure at the bottom shows the final decision regions, colored by category.

APPROXIMATIONS BY SERIES EXPANSIONS

In addition, considerable computation time may be required each time one of the methods is used to estimate *p(x)* or classify a new x.

In certain circumstances the Parzen-window procedure can be modified to reduce these problems considerably. The basic idea is to approximate the window function by a finite series expansion that is acceptably accurate in the region of interest.

If we can find two sets of functions $\varphi_j(x)$ and $X_j(x)$ that allow the expansion:

$$\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right)=\sum_{j=1}^{m} a_j \psi_j(\mathbf{x})\chi_j(\mathbf{x}_i),$$

then we can split the dependence upon $\mathbf{x}$ and $\mathbf{x}_i$ as

$$\sum_{i=1}^{n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right)=\sum_{j=1}^{m} a_j \psi_j(\mathbf{x})\sum_{i=1}^{n}\chi_j(\mathbf{x}_i).$$

$$p_n(\mathbf{x})=\sum_{j=1}^{m} b_j \psi_j(\mathbf{x})$$

where

$$b_j=\frac{a_j}{nV_n}\sum_{i=1}^{n}\chi_j(\mathbf{x}_i)$$

If a sufficiently accurate expansion can be obtained with a reasonable value for *m,* this approach has some obvious advantages. The information in the *n* samples is reduced to the *m* coefficients b_j.
If the functions $\varphi_j(.)$ and $X_j(.)$ are polynomial functions of the components of x and x_i, the expression for the estimate $p_n(x)$ is also a polynomial, which can be computed relatively efficiently.
Thus $\varphi((x-x_i)/h_n)$ should peak sharply at $x = x_i$ and should contribute little to the approximation of $p_n(x)$ for x far from x_i.
For simplicity, we confine our attention to a one-dimensional example using a Gaussian window function:

$$\sqrt{\pi}\,\varphi(u)=e^{-u^2}$$

$$\simeq\sum_{j=0}^{m-1}(-1)^j\frac{u^{2j}}{j!}.$$

This expansion is most accurate near $u = 0$, and is in error by less than $u^{2m}/m!$. If we substitute $u = (x - x_i)/h$, we obtain a polynomial of degree $2(m - 1)$ in x and x_i. For example, if $m = 2$ the window function can be approximated as

$$\sqrt{\pi}\,\varphi\left(\frac{x-x_i}{h}\right)\simeq 1-\left(\frac{x-x_i}{h}\right)^2$$

$$=1+\frac{2}{h^2}x\,x_i-\frac{1}{h^2}x^2-\frac{1}{h^2}x_i^2,$$

and thus

$$\sqrt{\pi}\,p_n(x)=\frac{1}{nh}\sum_{i=1}^{n}\sqrt{\pi}\,\varphi\left(\frac{x-x_i}{h}\right)\simeq b_0+b_1x+b_2x^2, \qquad (66)$$

where the coefficients are

$$b_0=\frac{1}{h}-\frac{1}{h^3}\frac{1}{n}\sum_{i=1}^{n}x_i^2$$

$$b_1=\frac{2}{h^3}\frac{1}{n}\sum_{i=1}^{n}x_i$$

$$b_2=-\frac{1}{h^3}.$$